

FMSV Course Project

LLM-Based Formal SVA Generation

GROUP 11

- **Aditya Bhawsar (230101006)**
- **Hemang Gautam (230101046)**
- **Jash Vaidya (230101049)**
- **Moksh Malhotra (230101066)**

Problem Statement & Motivation

Problem Statement

- Writing SystemVerilog Assertions (SVA) for formal verification requires deep RTL expertise and is highly time-consuming.
- Specification documents are verbose; manually mapping spec clauses to precise temporal SVA properties is error-prone.
- No automated pipeline exists that takes a natural language query + PDF spec + RTL and returns a formally proved assertion.

Objective

- Build an end-to-end pipeline: natural language query + PDF specification + RTL source → formally proved SVA assertion via SymbiYosys.
- Use LLMs to generate, validate, and iteratively refine assertions — without sending raw RTL code to the LLM.
- Represent the spec as a **modified KG tree** for minimal, targeted context retrieval at $O(N)$ cost with **minimal context guarantee**.

Tools / Tech Stack Used :

1. Groq api key as the primary llm (as 10 K free tokens allowed per account)
2. Groq 70B Cloud LLM
3. Sentence Transformer (local) for embedding generation.
4. Claude for generating queries.
5. SymbiYosis for formal verification.

Designs used :

1. SPEC: <https://github.com/hkust-zhiyao/AssertLLM/blob/main/spec/sockit.pdf>
RTL: <https://github.com/hkust-zhiyao/AssertLLM/tree/main/rtl/sockit>

Pipeline Overview (Phases 1 – 3)

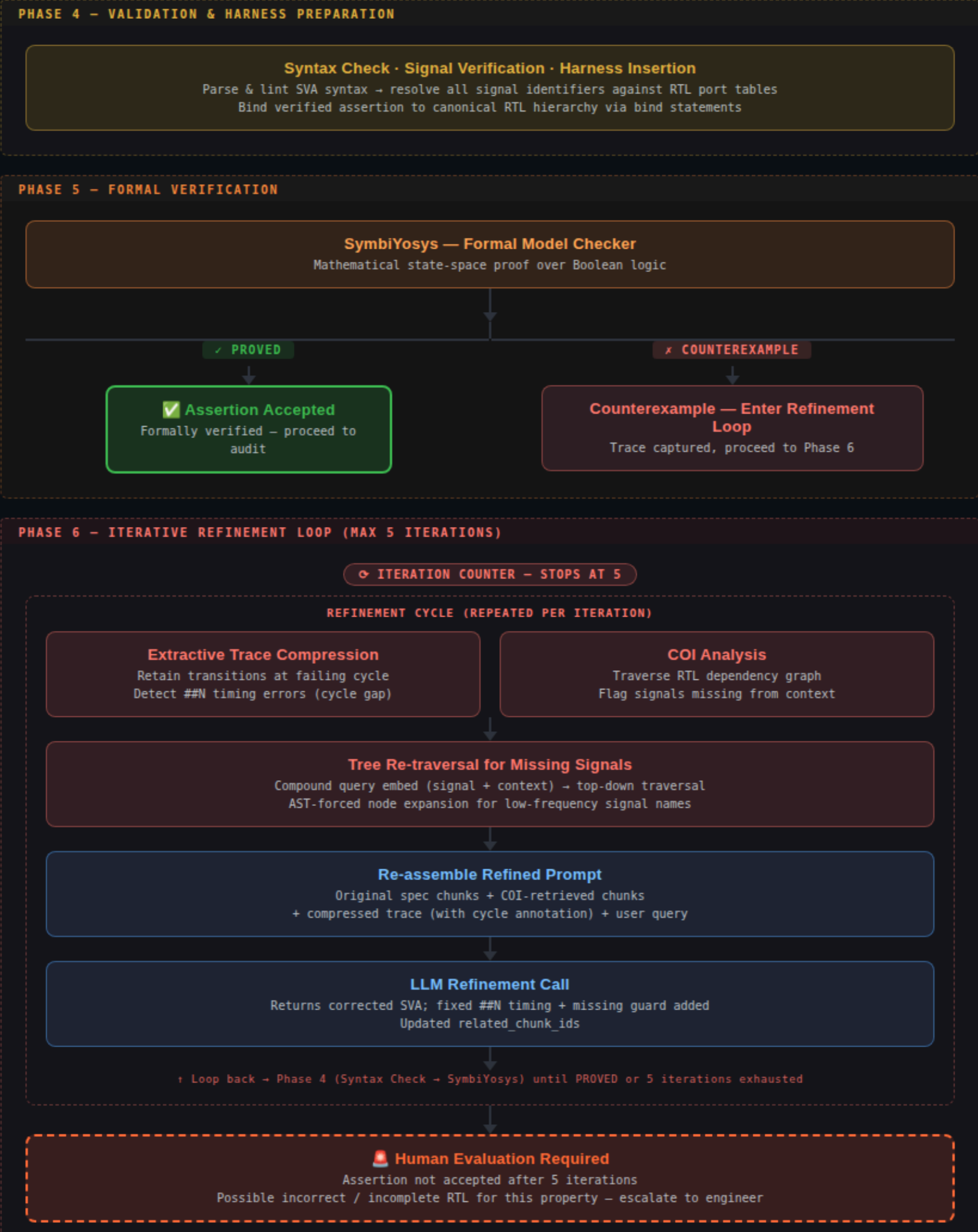
Note :

For preventing spurious passes due to rtl and spec signal mismatching, a symbol matching is performed later that checks that the signals present in assertion are strictly subset of those found in rtl in the rtl parsing phase.

=> Consistent signals



Pipeline Overview (Phases 4 – 6)



Complete Example

Step 1 : Node structure

```
[{"node_id": "L200_ff10e1",
  "is_leaf": false,
  "page_text": "",
  "chunk_summary": "This module selects one line from multiple OWN lines based on the select register,
                    and is part of a larger system that includes a 32-bit CPU with 1-wire modules, a task scheduler, CPU
                    interface, and shared resource management for improved signal quality and efficiency.",
  "embedding": [
    -0.07100147753953934,
    -0.0034632859751582146,
    -0.06042204424738884,
    0.03581060841679573,
    -0.033730506896972656,
    ...
  ],
  "signals": [
    "dat_r",
    "avalon_readdata",
    "read/write data slots",
    "reg",
    "OWN",
    ...
  ]
}]
```

Step 2 : Query

```
$ python main2.py test_rtl.v \
  --query "bus_rdt[4] equals owr_p[0] at all times when bus_adr[0] == 0.(bus_rdt_ctl_sts[4] is wired directly to owr_pwr[0] = owr_p[0].)" \
  --tree-file custom_tree.json \
  --depth 15 \
  --max-iterations 3 \
  --output final_result.json
```


Complete Example

Step 3 : Tree traversal by query
And minimal prompt construction

```
└─ [PARENT] L200_ff10e1 | sim=0.3104 ✓
└─ [LEAF ] L0_p111 | sim=0.1214 ✗ PRUNED
└─ [PARENT] L199_30ea22 | sim=0.4568 ✓
└─ [PARENT] L196_fe0d2d | sim=0.3011 ✓
└─ [LEAF ] L0_p179 | sim=0.2137 ✗ PRUNED
└─ [PARENT] L195_385904 | sim=0.3232 ✓
└─ [PARENT] L14_0b26fd | sim=0.1739 ✗ PRUNED
└─ [PARENT] L194_190d86 | sim=0.4100 ✓
└─ [LEAF ] L0_p197 | sim=0.2590 ✗ PRUNED
└─ [PARENT] L193_ec720f | sim=0.5024 ✓
└─ [PARENT] L188_b5c178 | sim=0.4965 ✓
└─ [LEAF ] L0_p48 | sim=0.4808 ✓
└─ [PARENT] L158_3b6587 | sim=0.3604 ✓
└─ [LEAF ] L0_p155 | sim=0.2753 ✓
└─ [PARENT] L128_7131d2 | sim=0.3818 ✓
└─ [LEAF ] L0_p151 | sim=0.3659 ✓
└─ [PARENT] L127_bb03a9 | sim=0.3356 ✓
└─ [LEAF ] L0_p156 | sim=0.2767 ✓
└─ [LEAF ] L0_p174 | sim=0.3397 ✓
└─ [PARENT] L192_edb291 | sim=0.3659 ✓
└─ [LEAF ] L0_p53 | sim=0.3173 ✓
└─ [PARENT] L191_be57cc | sim=0.3175 ✓
└─ [PARENT] L189_da777c | sim=0.2228 ✗ PRUNED
└─ [PARENT] L190_edd77c | sim=0.3334 ✓
└─ [PARENT] L169_289b33 | sim=0.3238 ✓
└─ [LEAF ] L0_p95 | sim=0.1850 ✗ PRUNED
└─ [PARENT] L165_a3b619 | sim=0.4166 ✓
└─ [LEAF ] L0_p106 | sim=0.2937 ✓
└─ [PARENT] L141_85817b | sim=0.4725 ✓
└─ [PARENT] L139_1cf0a5 | sim=0.4456 ✓
└─ [PARENT] L112_f106be | sim=0.3215 ✓
└─ [LEAF ] L0_p31 | sim=0.1991 ✗ PRUNED
└─ [PARENT] L98_ca17ba | sim=0.4029 ✓
└─ [LEAF ] L0_p11 | sim=0.4325 ✓
└─ [LEAF ] L0_p178 | sim=0.3179 ✓
└─ [PARENT] L138_eb09ea | sim=0.4953 ✓
└─ [LEAF ] L0_p0 | sim=0.4252 ✓
└─ [LEAF ] L0_p57 | sim=0.4835 ✓
└─ [PARENT] L140_2f5e78 | sim=0.4238 ✓
└─ [LEAF ] L0_p55 | sim=0.3667 ✓
└─ [LEAF ] L0_p56 | sim=0.4099 ✓
└─ [PARENT] L185_7f6dda | sim=0.2380 ✗ PRUNED
└─ [PARENT] L198_952321 | sim=0.4571 ✓
└─ [PARENT] L187_c6915f | sim=0.4146 ✓
└─ [LEAF ] L0_p17 | sim=0.3703 ✓
└─ [PARENT] L186_f37b94 | sim=0.3421 ✓
└─ [LEAF ] L0_p29 | sim=0.2801 ✓
└─ [PARENT] L184_b163b2 | sim=0.3130 ✓
└─ [PARENT] L174_40bac7 | sim=0.2297 ✗ PRUNED
└─ [PARENT] L183_19d4be | sim=0.3225 ✓
└─ [LEAF ] L0_p162 | sim=0.2919 ✓
└─ [PARENT] L182_768e3e | sim=0.2805 ✓
└─ [LEAF ] L0_p8 | sim=0.2364 ✗ PRUNED
└─ [LEAF ] L0_p146 | sim=0.2521 ✗ PRUNED
└─ [PARENT] L197_f1d06b | sim=0.3460 ✓
└─ [LEAF ] L0_p168 | sim=0.2204 ✗ PRUNED
└─ [LEAF ] L0_p176 | sim=0.3588 ✓

Leaves passing threshold : 17
Returning top-7
```

Complete Example

RTL ports analysis and 1st Iteration

-> DUT not given in first iteration, error reported to llm to correct in further iterations.

```
[Phase 3] Analysing RTL ports...
RTL top      : arbiter
Inputs       : ['bus_adr', 'bus_ren', 'bus_wdt', 'bus_wen', 'clk', 'owr_i', 'rst']
Outputs      : ['bus_irq', 'bus_rdt', 'owr_e', 'owr_p']
Data inputs: ['bus_adr', 'bus_ren', 'bus_wdt', 'bus_wen', 'owr_i']

-----
ITERATION 1 / 3
-----

[Step 4a] Generating assertion with LLM...
[LLM] Calling llama-3.3-70b-versatile (Attempt 1/10)...
RTL Port-Signal Table (auto-extracted – no RTL logic shown):
Signal      Dir      Width
-----
bus_adr      input    [0:0]
bus_ren      input    [0:0]
bus_wdt      input    [31:0]
bus_wen      input    [0:0]
clk          input    [0:0]
owr_i        input    [0:0]
rst          input    [0:0]
bus_irq      output    [0:0]
bus_rdt      output    [31:0]
owr_e        output    [0:0]
owr_p        output    [0:0]

CONSTRAINT: Every signal in assert() / assume() MUST appear in this table.
Do NOT invent signal names that are not listed above.
Assertion generated : module dynamic_checker (
    input wire [0:0] bus_adr,
    input wire [0:0] bus_ren,
    input wire [31:0] bus_wdt,
    input wire [0:0] bus_wen,
    input wire [0:0] clk,
    input wire [0:0] owr_i,
    input wire [0:0] rst,
    output wire [0:0] bus_irq,
    output wire [31:0] bus_rdt,
    output wire [0:0] owr_e,
    output wire [0:0] owr_p
);
    wire [0:0] owr_p_reg;
    assign owr_p_reg = owr_p;
    always @(posedge clk) begin
        if (bus_adr[0] == 0) begin
            assert(bus_rdt[4] == owr_p[0]);
        end
    end
endmodule
Pipeline depth : 1
Signals used    : ['bus_adr', 'bus_rdt', 'owr_p', 'clk']
Reasoning       : The assertion checks that bus_rdt[4] equals owr_p[0] at all times when bus_adr[0] equals 0. This is based on the provided specification context, which states that bus_rdt_ctl_sts[4] is wired directly to owr_pwr[0] = o
wr_p[0]. The checker uses a simple always block to check the assertion at every positive clock edge.

[Step 4b] 🛑 Hard pre-flight FAILED – blocking sby run:
  • Missing DUT instantiation: 'arbiter DUT (...)' not found. Without the DUT, output signals are free unconstrained variables and the checker verifies nothing meaningful. You MUST include: arbiter DUT (.clk(clk), .rst(rst), ...all port
s...);
```


Complete Example

Final accepted assertion at 3rd iteration and yosys simulation.

```
-----
ITERATION 3 / 3
-----

[Step 4a] Generating assertion with LLM...
[LLM] Calling llama-3.3-70b-versatile (Attempt 1/10)...
RTL Port-Signal Table (auto-extracted – no RTL logic shown):
Signal          Dir      Width
-----
bus_adr          input    [0:0]
bus_ren          input    [0:0]
bus_wdt          input    [31:0]
bus_wen          input    [0:0]
clk              input    [0:0]
owr_i            input    [0:0]
rst              input    [0:0]
bus_irq          output    [0:0]
bus_rdt          output    [31:0]
owr_e            output    [0:0]
owr_p            output    [0:0]

CONSTRAINT: Every signal in assert() / assume() MUST appear in this table.
Do NOT invent signal names that are not listed above.
Assertion generated : module dynamic_checker (
    input wire [0:0] bus_adr,
    input wire [0:0] bus_ren,
    input wire [31:0] bus_wdt,
    input wire [0:0] bus_wen,
    input wire [0:0] clk,
    input wire [0:0] owr_i,
    input wire [0:0] rst,
    output wire [0:0] bus_irq,
    output wire [31:0] bus_rdt,
    output wire [0:0] owr_e,
    output wire [0:0] owr_p
);
    arbiter DUT (
        .bus_adr(bus_adr),
        .bus_ren(bus_ren),
        .bus_wdt(bus_wdt),
        .bus_wen(bus_wen),
        .clk(clk),
        .owr_i(owr_i),
        .rst(rst),
        .bus_irq(bus_irq),
        .bus_rdt(bus_rdt),
        .owr_e(owr_e),
        .owr_p(owr_p)
    );
    reg initialized = 0;
    always @(posedge clk) begin
        initialized <= 1;
        assume(rst == !initialized);
        if (initialized) begin
            if (bus_adr[0] == 0) begin
                assert(bus_rdt[4] == owr_p);
            end
        end
    end
endmodule
```

```
[Step 4c] Pre-flight signal validation...
All signals validated ✓

[Step 4d] Running SymbiYosys formal check...
[sby] Running SymbiYosys (depth=15, top=arbiter)...
SBY 21:49:49 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: smtbmc z3
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Solver: z3
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 0..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 0..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 1..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 1..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 2..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 2..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 3..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 3..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 4..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 4..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 5..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 5..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 6..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 6..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 7..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 7..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 8..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 8..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 9..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 9..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 10..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 10..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 11..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 11..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 12..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 12..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 13..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 13..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assumptions in step 14..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Checking assertions in step 14..
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: ## 0:00:00 Status: passed
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] engine_0: Status returned by engine: pass
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] summary: Elapsed clock time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] summary: Elapsed process time [H:MM:SS (secs)]: 0:00:00 (0)
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] summary: engine_0 (smtbmc z3) returned pass
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] summary: engine_0 did not produce any traces
SBY 21:49:50 [/Users/hemanggautam/Desktop/fmsv_new/testfiles/check] DONE (PASS, rc=0)

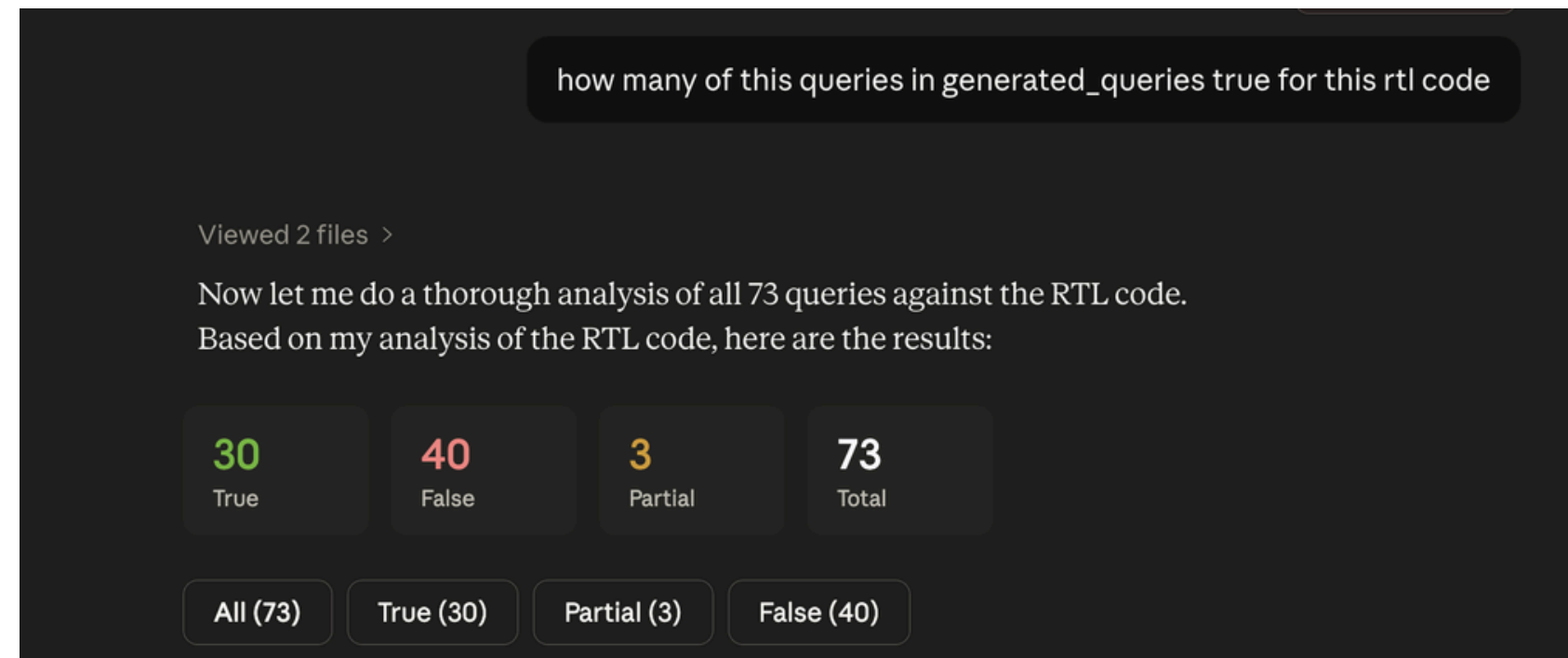
=====
✓ PASS – all assertions hold

Status : PASS

=====
✓ ASSERTION VERIFIED – PIPELINE COMPLETE
=====
```


Key Limitations and Challenges Faced

- We tried to generate queries automatically using Groq llm.
- **Method used :**
 1. Give spec in chunks.
 2. In each chunk, identify a signal, try to form all possible properties using that chunk.
- Then, We manually ask Claude to review those properties as per the rtl code



- However, the results were **not satisfactory**.
- Hence generated properties/queries for the rtl were generated manually using Claude and we focused on **minimal context retrieval** for those queries to be proven by a formal verification tool .

Key Limitations and Challenges Faced

- **Yosys Free Version Syntax Limits** : Rejects temporal operators (e.g., `|->`, `##N`) and wildcard binding (`.*`).
- **LLM Logic Errors** : Groq frequently misunderstands negation properties.
- **Operator Misuse** : The LLM consistently formats the `$past()` operator incorrectly.
- **No Bindings**: Groq often drops all bindings in bind statements.

We tried to solve these issues by giving some strict rules in prompt. However,

- **Prompt Non-Compliance**: As a free model, Groq repeatedly ignores strict prompt constraints, causing many queries to fail.

Key LLM Prompt — assertion_generator.py (System Prompt)

You are an expert in formal hardware verification using SymbiYosys (sby) + Yosys.

{port_table} <- Resolved concrete port widths injected here. No raw RTL, no parameter names.

QUERY POLARITY - READ CAREFULLY

{polarity_block} <- POSITIVE: assert must FAIL if event does NOT occur.

 <- NEGATIVE (never / does not): assert must FAIL if prohibited event DOES occur.

ABSOLUTE PROHIBITION: THE -> IMPLICATION OPERATOR

WRONG: assert(A -> B);

CORRECT: assert(!A || B); OR if (A) begin assert(B); end

YOSYS-COMPATIBLE SYNTAX - ALLOWED inside always @(posedge clk):

 assert(<bool>); assume(<bool>); \$past(signal, N); \$initstate; if/else; reg counters

FORBIDDEN (Yosys parse error - DO NOT USE):

 assert property(...) |-> |=> ##N [*N] [->N] -> \$rose() \$fell() \$stable()

 disable iff(...) @(posedge clk) inside assert/assume bind keyword .* wildcard

PORT WIDTH RULES - CRITICAL:

Every signal MUST declare exact bit-width from port table. Never use parameter names.

CORRECT: input wire [31:0] bus_wdt, WRONG: input wire [BDW-1:0] bus_wdt

\$past() INDEXING - HARD SYNTAX RULE:

WRONG: assert(out[7] == \$past(in,1)[7]);

CORRECT: assert(out[7] == \$past(in[7],1));

MODULE STRUCTURE (STRICT ORDER):

1. All inputs as "input wire [W:0] sig," in port list

2. All outputs as "wire [W:0] sig;" inside module body

3. DUT: arbiter DUT (.clk(clk), .rst(rst), ...ALL ports...); <- MANDATORY

4. Verification body: reg initialized=0; always @(posedge clk) begin ... end

OUTPUT FORMAT - respond ONLY with this JSON (no markdown, no backticks):

```
{ "assertion_sv": "module dynamic_checker (\n ...",
  "signals_used": ["clk","rst",...],
  "pipeline_depth": <int>,
  "reasoning": "<one paragraph>" }
```

Local LLM/SLM

Model	Coding Capabilities	<code>bind</code> Construct	<code>assume</code> vs <code>assert</code>	Cycle Guards	Verdict
Phi-4-mini (3.8B)	❌ Partial	❌ Likely missing	❌ Often confused	❌ Dropped	❌ Not suitable
Qwen2.5-7B-Coder	✅ Usually correct	⚠️ Sometimes	✅ Good	⚠️ Sometimes	🟡 Borderline
Qwen2.5-Coder-32B	✅ Complete	✅ Correct	✅ Strong	✅ Yes	✅ Best local
DeepSeek-R1-8B	✅ Good	⚠️ Sometimes	✅ Good reasoning	✅ Usually	🟢 Good budget option

We evaluated four local LLMs/SLMs on their ability to generate valid SVA checkers, focusing on four key criteria: coding capabilities, bind construct usage, assume vs assert distinction, and cycle guard handling.

- Phi-4-mini (3.8B) struggled across all dimensions — coding capabilities, missing binds, confused assume/assert usage, and **dropped cycle guards**. Not suitable for this task.
- Qwen2.5-7B-Coder showed promise with usually correct codes and good assume/assert handling, but was inconsistent with **binds and cycle guards**.
- Qwen2.5-Coder-32B required around **20GB VRAM**. So we were unable to test it.
- DeepSeek-R1-8B demonstrated good reasoning and usually correct behavior, but was **inconsistent with bind constructs**.

DeepSeek-R1-8B

```
time=2026-04-22T22:30:14.309+05:30 level=INFO source=device.go:240 msg="model weights" device=CUDA0 size="2.5 GiB"
time=2026-04-22T22:30:14.309+05:30 level=INFO source=device.go:245 msg="model weights" device=CPU size="2.4 GiB"
time=2026-04-22T22:30:14.309+05:30 level=INFO source=device.go:251 msg="kv cache" device=CUDA0 size="704.0 MiB"
time=2026-04-22T22:30:14.310+05:30 level=INFO source=device.go:256 msg="kv cache" device=CPU size="448.0 MiB"
time=2026-04-22T22:30:14.310+05:30 level=INFO source=device.go:262 msg="compute graph" device=CUDA0 size="182.0 MiB"
time=2026-04-22T22:30:14.310+05:30 level=INFO source=device.go:267 msg="compute graph" device=CPU size="8.0 MiB"
time=2026-04-22T22:30:14.310+05:30 level=INFO source=device.go:272 msg="total memory" size="6.2 GiB"
time=2026-04-22T22:30:14.311+05:30 level=INFO source=sched.go:561 msg="loaded runners" count=1
time=2026-04-22T22:30:14.311+05:30 level=INFO source=server.go:1364 msg="waiting for llama runner to start responding"
time=2026-04-22T22:30:14.312+05:30 level=INFO source=server.go:1398 msg="waiting for server to become available" stat
"llm server loading model"
time=2026-04-22T22:30:16.832+05:30 level=INFO source=server.go:1402 msg="llama runner started in 3.78 seconds"
[GIN] 2026/04/22 - 22:40:37 | 200 | 10m27s | 127.0.0.1 | POST | "/api/chat"
```

- Despite showing strong reasoning capability in benchmarks, DeepSeek-R1-8B proved impractical in our setup. As seen in the server logs, the model loaded approximately 2.5 GiB of weights onto the GPU with an additional 2.4 GiB on CPU, consuming over **6.2 GiB of total memory**.
- The llama runner took 3.78 seconds just to start, and the system was unable to serve responses reliably under our hardware constraints. This ruled it out as a practical local deployment option despite its theoretical capability.

Qwen2.5-7B-Coder

```
=====
✅ ASSERTION VERIFIED – PIPELINE COMPLETE
=====

Final Assertion:

module dynamic_checker (
    input wire clk,
    input wire rst,
    input wire [31:0] bus_wdt,
    input wire owr_i,
    output wire bus_irq
);
    wire owr_e;
    assign owr_e = !(bus_wdt[1:0] == 2'b11);

    arbiter DUT (
        .clk(clk),
        .rst(rst),
        .bus_adr(bus_adr),
        .bus_ren(bus_ren),
        .bus_wdt(bus_wdt),
        .bus_wen(bus_wen),
        .owr_i(owr_i),
        .bus_irq(bus_irq)
    );

    reg initialized = 0;
    always @(posedge clk) begin
        initialized <= 1;
        assume(rst == !initialized);

        if (initialized && bus_wdt[1:0] == 2'b11) begin
            assert(owr_e == 0);
        end
    end
endmodule
```

```
=====
✅ ASSERTION VERIFIED – PIPELINE COMPLETE
=====

Final Assertion:

module dynamic_checker (
    input wire clk,
    input wire rst,
    input wire bus_adr,
    input wire bus_ren,
    input wire [31:0] bus_wdt,
    input wire bus_wen,
    output wire bus_irq,
    output wire [31:0] bus_rdt,
    output wire owr_e,
    output wire owr_p,
    input wire owr_i
);
    wire initialized = 0;
    arbiter DUT (
        .clk(clk),
        .rst(rst),
        .bus_adr(bus_adr),
        .bus_ren(bus_ren),
        .bus_wdt(bus_wdt),
        .bus_wen(bus_wen),
        .bus_irq(bus_irq),
        .bus_rdt(bus_rdt),
        .owr_e(owr_e),
        .owr_p(owr_p),
        .owr_i(owr_i)
    );

    always @(posedge clk) begin
        initialized <= 1;
        assume(rst == !initialized);

        if (initialized && bus_adr == 0) begin
            assert(bus_rdt[5] == 0);
        end
    end
endmodule
```

- Qwen2.5-7B-Coder was lightweight enough to run on our systems, but the quality of generated assertions was consistently poor.
- In both cases, **port signal bindings were frequently skipped** and the **assume/assert relationship was misused**, leading to assertions that pass trivially rather than proving meaningful properties.

```

Assertion generated : module dynamic_checker (
  input wire clk,
  input wire rst,
  output reg owr_e
);
always @(posedge clk) begin
  if (rst) begin
    owr_e <= 1'b0;
  end else begin
    // Additional assertions can be added here
  end
end
end

```

```

Assertion generated : module dynamic_checker (
  input wire clk,
  input wire rst,
  output reg [31:0] bus_adr,
  output reg [31:0] bus_wdt,
  output reg [0:0] bus_wen,
  inout wire [7:0] owr_i,
  output reg [0:0] owr_e
);
always @(posedge clk) begin
  if (rst) begin
    owr_e <= 1'bz;
  end else begin
    // Additional logic to set owr_e based on t
    // This assertion checks that owr_e is high
    or reset.
    assert(!rst || (owr_e == 1'bz));
  end
end
end

```

Phi-4-mini (3.8B)

- Phi-4-mini at 3.8B parameters was the weakest performer. The generated outputs show two fundamental failure modes:
- The top example produces a **near-empty module** — only a reset condition setting owr_e to 1'b0, with a placeholder comment instead of any real assertion logic. **No bind, no DUT instantiation, no meaningful check.**
- The bottom example **incorrectly declares DUT output signals** as output reg and inout ports in the checker module itself, confusing the checker's interface with the DUT's. The assertion `assert(!rst || (owr_e == 1'bz))` checks for high-impedance, which is semantically incorrect for this design.

Future improvements:

- **RTL and spec signal alignment.**
- Refinement targets signal coverage and trace understanding — not semantic alignment with the query. The loop augments context (COI-identified missing signals) but does not re-evaluate whether the new assertion still reflects the original property intent.
- **No local LLM.** All calls go to Groq cloud API, introducing **rate-limit failures** (mitigated by **API key rotation** across API_list.txt), latency variance (8-293 s), and dependency on external availability.
- No cross-query memory. Every query starts from scratch — **previously proved properties are not reused to constrain** or guide new assertion generation. Will be **improved by using local LLM** and accessing its memory.

Research Questions — P1: Long-Context Problem

RQ1.1 — How do we select a minimal context subset that preserves correctness?

- Hierarchical binary spec tree with top-down cosine-similarity traversal at $O(\log N)$.
- Returns only **top-k=7 leaf chunks** relevant to the query — no raw spec or full AST sent to LLM.

RQ1.2 — How does the system detect and fill missing context?

- Phase 6: Yosys **COI analysis** identifies RTL signals absent from current context.
- **Tree is re-traversed** with a compound query (signal + context) to retrieve missing chunks.

RQ1.3 — What representations best preserve task-critical information?

- Dense embeddings (all-MiniLM-L6-v2) for similarity-based leaf retrieval. **(LOSSY SUMMARY)**
- LLM-generated summaries at internal nodes for fast branch-level pruning.
- Structured port tables (lossless, deterministically parsed) — no raw RTL ever included. **(LOSSLESS SUMMARY)**

Research Questions — P2: Agentic Memory

RQ1.4 — What memory representation yields best retrieval fidelity and update stability?

- Three-layer hybrid memory architecture:
- Leaf nodes: **exact spec content** in small chunks - context is fully as per spec
- Internal nodes: **LLM-generated summaries (lossy)** — fast branch pruning without full leaf descent.
- Port tables: **deterministically** parsed from RTL, **lossless, zero LLM involvement** — used where signal correctness is non-negotiable.

RQ1.5 — How do we prevent memory poisoning, staleness, or contradictions?

- Tree built once offline from spec; serialised to custom_tree.json — **read-only at query time**. No LLM output can corrupt stored embeddings.
- **Polarity detection** before every prompt **prevents assertion direction from flipping** across iterations — contradictions structurally impossible.
- **Staleness** bounded to spec version changes only; not per-query.

RQ1.6 — Can memory be task-adaptive?

- Iteration 1: **retrieval is query-driven** — top-k=7 chunks closest to the natural language property.
- On verification failure: query recomposed as (original query + COI-flagged missing signals) — tree returns **signal-specific chunks** instead of broad property context.
- Memory access pattern changes as a function of verification outcome — **generation vs. debugging tasks fetch different granularities automatically**.

Research Questions — P3: Cloud LLM Efficiency

RQ1.7 — How do we minimize tokens sent per request without losing quality?

- Send only: resolved port signal table (~few hundred tokens) + top-k spec summaries (~700 tokens).
- Raw RTL, full ASTs, and **complete spec docs are never transmitted**.
- Core principle: bandwidth-minimal payload.

RQ1.8 — Lossy summaries vs. lossless structured state — which is safe?

- Spec **chunk summaries: lossy** (LLM-generated) — sufficient for property context.
- Port **signal table: lossless** (deterministically parsed from RTL) — required for signal correctness.
- Counterexample traces: extractive compression — retain only failing cycle + ##N timing gaps.

Research Questions — P4: Prompt Mapping

RQ2.4 — When can a new prompt reuse cached intermediate results safely?

- Spec tree (201 pre-embedded chunks) built once; all queries traverse the same cached tree — no re-embedding per query.
- Reuse gated by cosine similarity $\theta = 0.25$ — chunks below threshold excluded, preventing stale context injection.
- Port table parsed once at iteration 1; reused across all refinement iterations — safe because RTL structure is invariant within a query.

RQ2.5 — What is the right granularity of caching?

- Two levels simultaneously:
- Chunk-level (35-word spec chunks + embeddings) — reused across all queries on the same spec.
- Structured-state-level (port table) — reused across all refinement iterations of one query.
- Prompt-level caching is unsafe — polarity block, counterexample hints, and COI signals change every iteration.

RQ2.6 — How do we validate cached results to prevent error propagation?

- Three-layer validation before any cached artefact reaches the LLM:
- Cosine gate — similarity ≥ 0.25 required for chunk inclusion.
- Signal subset check — assertion signals must be strict subset of cached port table; violations rejected before SymbiYosys runs.
- Formal verification as ground truth — cached counterexample traces re-injected only if consistent with current assertion's COI signal set

Research Questions — P6: Feedback-Driven Refinement

RQ3.1 — What feedback signals are available?

- SymbiYosys counterexample traces (VCD) — exact signal values at the failing cycle.
- Yosys parse errors — line numbers + offending code, machine-readable, no human annotation.

RQ3.2 — How is feedback turned into improved assertions?

- Counterexample values + COI-flagged missing signals injected into next iteration's prompt.
- Yosys errors include an exact fix directive (e.g. `assert(!A || B)` instead of `assert(A->B)`).
- Tree re-traversed to augment context with newly-identified relevant spec chunks.

RQ3.3 — How is drift and runaway refinement prevented?

- Hard cap of 5 iterations — loop never runs indefinitely.
- Polarity detection before every prompt ensures LLM cannot flip assertion direction.
- Human escalation triggered on exhaustion — no unproved assertion is silently accepted.

Results Summary — 24 Property Queries on SocKit OWM

24

Total Queries

8

PROVED (33%)

13

No Convergence

3

Pipeline Error

What passed:

- Q1-Q3: Reset assertions (owr_e, owr_p, bus_irq) — proved in 1 iteration (~19-27 s each).
- Q5, Q6: ctl_sts write -> bus_irq low; bus_rdt[5] hardwired to 0 — proved in 1 iteration.
- Q12: bus_rdt[4] == owr_p[0] — required iteration 2; COI retrieved additional context that was absent in iteration 1.
- Q18, Q24: owr_i bus isolation (no combinatorial path to bus_irq, owr_e, owr_p) — proved in 1 iteration.

What failed (recurring pattern):

- Queries involving one-cycle delayed write-to-read semantics (Q7-Q11, Q13-Q17, Q19-Q20) consistently failed.
- Root cause: LLM used `$past(signal,1)` to check the previous value instead of asserting the current (next-cycle) value — inverted temporal direction.
- Refinement loop correctly identified missing signals via COI but did not fix the `$past()` misuse, leading to 5-iteration exhaustion.
- 3 queries (Q21-Q23) hit pipeline errors (Groq rate-limit / sby subprocess timeout, 241-293 s).

Average time per query: ~60 s (range: 8 s - 293 s). Queries that passed averaged ~27 s.

Results

Per-query summary:

#	Status	Iters	Leaves%	Time(s)	Fail reason
1	✓ PASS	1	3.5%	26.5	—
2	✓ PASS	1	3.5%	19.4	—
3	✓ PASS	1	3.5%	18.6	—
4	✗ FAIL_NO_CONVERGENCE	5	3.5%	24.0	no_convergence
5	✓ PASS	1	3.5%	19.2	—
6	✓ PASS	1	3.5%	16.2	—
7	✗ FAIL_LOGIC_ERROR	5	1.0%	25.0	no_convergence
8	✗ FAIL_LOGIC_ERROR	5	0.5%	26.5	no_convergence
9	✗ FAIL_LOGIC_ERROR	5	3.5%	26.8	no_convergence
10	✗ FAIL_LOGIC_ERROR	5	2.0%	35.9	no_convergence
11	✗ FAIL_LOGIC_ERROR	5	3.5%	54.7	no_convergence
12	✓ PASS	2	3.5%	47.3	—
13	✗ FAIL_LOGIC_ERROR	5	0.5%	75.2	no_convergence
14	✗ FAIL_LOGIC_ERROR	5	3.5%	74.7	no_convergence
15	✗ FAIL_LOGIC_ERROR	5	3.5%	67.8	no_convergence
16	✗ FAIL_LOGIC_ERROR	5	0.5%	79.2	no_convergence
17	✗ FAIL_LOGIC_ERROR	5	3.5%	112.8	no_convergence
18	✓ PASS	1	3.5%	43.6	—
19	✗ FAIL_LOGIC_ERROR	5	3.0%	241.2	no_convergence
20	✗ FAIL_LOGIC_ERROR	3	1.0%	204.2	counterexample
21	✗ UNKNOWN	1	0.5%	293.4	pipeline_error
22	✗ UNKNOWN	0	2.5%	8.5	pipeline_error
23	✗ UNKNOWN	1	3.5%	109.5	pipeline_error
24	✓ PASS	1	0.5%	28.8	—

Codebase — File-by-File Overview (1/2)

extract_spec.py

Reads the PDF specification (socket.pdf), strips headers/footers, produces clean flat text (spec_clean.txt) for tree building.

spec_tree.py

Chunks spec text (35 words/chunk, 7-word overlap), embeds each chunk via all-MiniLM-L6-v2, builds bottom-up binary similarity tree. Internal nodes store LLM-generated summaries (Llama 3.1-8B).

custom_tree.json

Serialised offline output — 201 leaf nodes with embeddings, summaries, and signal lists. Loaded at query time to avoid re-embedding.

tree_retriever.py

Top-down $O(\log N)$ traversal. Cosine-similarity prunes branches below threshold (0.25). Returns top-k ($k=7$) most relevant leaf chunks for a given query.

rtl_utils.py

Pure static RTL analysis — parses port declarations, runs Yosys COI (Cone-of-Influence) on counterexample traces, extracts signal identifiers. No LLM calls here.

assertion_generator.py

Core orchestrator: builds the full system + user LLM prompt (with polarity detection, port table injection, counterexample hints), calls Groq, parses JSON response, runs all pre-flight checks.

llm_client.py

Thin HTTP wrapper for Groq Chat Completions API. All LLM calls route through here. Temperature fixed at 0.1 for deterministic output.

formal_runner.py

Orchestrates SymbiYosys: writes .sv + .sby files to temp dir, runs sby subprocess, parses PASS/FAIL/ERROR, reads VCD counterexample traces, runs COI on failure.

Codebase — File-by-File Overview (2/2)

result_builder.py

Constructs standardised result dicts from sby output. Generates natural-language improvement suggestions for the next refinement iteration.

main1.py

Offline entrypoint — builds and serialises the hierarchical spec tree. Run once per specification document.

main2.py

Online entrypoint — runs the full 6-phase pipeline for a single natural language query and prints the result.

batch_eval.py

Runs all 24 queries sequentially, collects results, and writes report.json with per-query status, timing, and iteration counts.

config.py

Central configuration: model names, API key, chunk size (35 words), retrieval k (7), cosine threshold (0.25), BMC depth (10), max iterations (5), parameter-to-concrete width map.

queries.txt

The 24 natural language property descriptions used for evaluation — covering reset, register read/write, and bus isolation behaviour.

report.json

Final evaluation output — per-query status (PASS / FAIL_LOGIC_ERROR / FAIL_NO_CONVERGENCE / UNKNOWN), iterations used, elapsed time, and final assertion SV text.

test_rtl.v / sockit_owm.v

The target RTL design — SockKit 1-Wire Master. Deterministically parsed for port tables; never sent to the LLM as raw source.

sockit.pdf

Input specification document. Processed by extract_spec.py into spec_clean.txt, then chunked and embedded by spec_tree.py.

Conclusion

- Context required by each query was greatly reduced.
- Using better llm's , we would be reaching higher accuracy because majorith of the generated assertions are failing because of inability of free llm's to follow prompt strictly.